6809 Forth — Open Items Checklist

Everything below was explicitly flagged during the build as incomplete, unverified, or deliberately deferred. Nothing here is a surprise — each item was named at the point it came up. This is a consolidated list to work from, not a new set of findings. Regenerated to reflect fixes and design decisions made since the original version.

Known bugs — resolved since the original checklist

- **`DUMP` 's partial-final-line bug** — fixed via `DUVALID` tracking (`25_tools_word_set.asm`). The ASCII column no longer reads past the valid bytes on a short final line.
- **`SUBSTITUTE`** — completed. It now performs a real bounds-checked copy (prefix / replacement / suffix) via a shared `SUBCOPY` helper, reusing `SEARCHW` for the match. Still scoped to a single registered name/value pair, not a full table — see "Open design questions" below; that scope limit is a deliberate simplification, not a bug.
- **`VALUE` 's PFA** — moved from `CODEHERE` (immutable space) to `VARHERE` (mutable space), matching `VARIABLE` 's pattern, so every `TO` -driven update now writes into the correct region instead of into code space. Not on the original checklist (found and fixed after this list was first generated); logged here for the record.
- **Every scratch/global cell now has a real `RMB` -assigned address in page zero** (`00_memory_map_and_globals.asm`), applied rather than left as a placeholder. Two real findings came out of doing this: the budget is **253 of 256 bytes used — only 3 bytes of headroom** in the GLOBALS page, and `SNEND` (documented in the original placeholder list) turned out to be genuinely dead — never read or written anywhere — and was dropped rather than given an address. Any future scratch cell added to this system will need to fit in that remaining 3 bytes or the page-zero fast-addressing property (DP = `$00`) stops covering it.
- **The ROM base dictionary is now real** (`27_forth_dictionary.asm`) — every primitive with actual code got a real header, chained via `LINK` , `CFA` pointing straight at its code label. `ABORTHDR` 's `LINK` field, a placeholder `0` since it was first built, is now resolved to the chain's newest entry. Building this surfaced two real findings, not just mechanical work: the original 1024-byte `BASEDICT` could not hold the header table (ultimately 1954 bytes needed, once `DOES>` was added — see below), so it was resized to 2048 bytes, taking the space from `BASECODE` (now 6080 bytes, was 7104) — a resize based purely

on the header-table budget, not on any actual measurement of `BASECODE`'s assembled size, which has never been checked with a real 6809 assembler. Generating this table also surfaced that **`DOES>` had no corresponding code anywhere in the file** — resolved in a follow-up pass, see below.

- **`DOES>` — resolved.** `SETDOES` (the patching runtime) was added beside `DODOES` / `DOESRT0`, and `DOES>` itself (code label `DOESGT`, since a literal `">"` is not a valid 6809 assembler label) was added right after `CREATE`. `DOESBEH` was added to the GLOBALS layout to support it, using 2 of the page's last 3 free bytes — only 1 byte of headroom now remains in page zero. `DOES>` also has a real dictionary header now (`H_DOESGT`, the chain's newest entry); `ABORTHDR`'s `LINK` was updated again to point at it.
- **Four internal loop-label names were reused across unrelated routines: `FLOOP`, `UDLOOP`, `DRPOS`, `CMLOOP`.** Resolved — each pair renamed to a distinct name scoped to its own routine: `FIND`'s `FLOOP` → `FFLOOP`; `FILLW`'s `FLOOP` → `FILLOOP`; `UDIV16`'s `UDLOOP` → `UD16LOOP`; `UDDIGIT`'s `UDLOOP` → `UDDLOOP`; `DIVCOMMON`'s `DRPOS` → `DCRPOS`; `DDOTR`'s `DRPOS` → `DDRPOS`; `CMOVEW`'s `CMLOOP` → `CMVLOOP`; `COMPAREW`'s `CMLOOP` → `CMPLOOP`. Verified zero duplicate labels and zero stray references to any of the old names remain anywhere in the file.
- **Hardware (RTS) serial handshaking — implemented.** `IRQH`'s receive path and `KEY` now toggle RTS based on the input ring's fill level (`INFILL`, `RTSCHECKHI`, `RTSCHECKLO` against `INHIWATER`=48/`INLOWATER`=16, with hysteresis between them) via a new `CR_RTSHI` control byte and `RTSSTATE` flag — the last free byte in the GLOBALS page, which is now fully packed at 256/256. CTS (the other handshaking direction) needed no firmware logic at all: confirmed against the 6850 datasheet that TDRE is automatically inhibited while CTS is deasserted, so this system's existing TDRE-gated transmit logic already respects it. A real race was identified and closed: mainline code (`KEY`, via `RTSCHECKLO`) and the ISR (`IRQH`'s `TXOFF`) can both decide to write the control register, so `RTSCHECKLO` masks IRQ around its critical section and `TXOFF` checks `RTSSTATE` before writing. Software (XON/XOFF) handshaking remains unimplemented — see below.
- **`TRUE` and `FALSE` — implemented.** Surfaced while organizing the ANS test suite: neither existed as an actual dictionary word, only as the internal `TRUEV` / `FALSEV` assembler constants. Added as `CONSTANT TRUE -1` / `CONSTANT FALSE 0` (`TRUEBODY` / `FALSEBODY`, section 26; headers `H_TRUE` / `H_FALSE`, chain's two newest entries, section 27) — the first `CONSTANT`-pattern ROM-resident words in this system. Every other ROM word's CFA is a plain code label; a `CONSTANT`'s CFA is the `DODOES`-trampoline pattern instead, built here with fixed, assemble-time addresses rather than a `CODEHERE` snapshot, since there is no interactive `CREATE` / `CONSTANT` phase for ROM content.

- **One genuine 6309-only instruction (`TSTD`) was in use — fixed.** Found by auditing every mnemonic in the file against the real 6809 instruction set, rather than trusting the Build Instructions section's earlier claim that no 6309 extensions were used (that claim was wrong until this fix). `TRYNUM` (section 9) used `TSTD` with no operand to test whether `D` was zero after `PULU D` — `PULU` / `PULS` don't affect condition codes on genuine 6809 hardware, unlike a load, so this relied on a 6309-only convenience instruction. Replaced with `CMPD #0` , a real 6809 instruction with identical behavior for this purpose. A full mnemonic audit after the fix found zero remaining non-6809 instructions anywhere in the file.

- **`BASELATEST` was an undefined symbol — a real assembly-breaking bug, not a placeholder.** `COLD` (section 4) has referenced `LDD #BASELATEST` to initialize `LATEST` since this file's earliest version, and the SECTION 27 header comment has long asserted " `BASELATEST` remains `QUITHDR` " - but no `EQU` or label actually named `BASELATEST` existed anywhere in the file. This would have failed to assemble outright. Fixed by adding `BASELATEST EQU QUITHDR` directly after `QUITHDR` 's own definition (section 26), matching what the comment always said it should equal. Verified: exactly one definition, no duplicate symbols, and `COLD` 's forward reference to it resolves under standard two-pass assembly.

- **`JSR CR` (bare, undefined) appeared at five call sites — a real assembly-breaking bug, not a naming inconsistency.** The actual routine has always been labeled `CRW` (section 22), following this file's own convention of giving short/symbolic ANS names a distinct code label. `ABORT` , `QUIT` 's error-report path, `QUIT` 's `ok` -prompt path, `WORDS` (`WWDONE`), and `DUMP` (`DULEND`) all called the bare, undefined `CR` instead. Fixed at each site to `JSR CRW` , preserving each call site's original spacing exactly. Verified: `CRW` defined exactly once, referenced 7 times total, zero remaining bare- `CR` instruction operands anywhere in the file (the two harmless bare "CR" occurrences that remain are a section-title comment and the dictionary header's `FCC "CR"` name string, neither of which is a bug).

- **Two ALU instructions used invalid register-to-register syntax (`ADDA B` , `EORA B`) — the 6809 has no such addressing mode, so the assembler read `B` as an undefined direct-page symbol.** Found in the single-cell multiply routine (`ADDA B` , twice) and `DOPLUSTEST` 's sign comparison for `+LOOP` (`EORA B`). Fixed with the standard 6809 idiom for combining two registers: `PSHS B` followed by `ADDA ,S+` / `EORA ,S+` (push B, then operate through the auto-incrementing stack-indexed operand). Verified: a full sweep of every ALU instruction (`ADDA` / `ADDB` / `SUBA` / `SUBB` / `ANDA` / `ANDB` / `ORA` / `ORB` / `EORA` / `EORB` / `CMPA` / `CMPB` / `ADCA` / `ADCB` / `SBCA` / `SBCB` / `BITA` / `BITB`) against a bare `A` / `B` operand found zero remaining instances; every other bare `B` in the file (in `STA` / `LDA` / `LEAX ...,X` and `PSHS B`) is genuine, valid 6809 accumulator-offset

indexed addressing or register-list syntax.

- **Nine scratch symbols (`MRESULT` , `MVCNT` , `MVDST` , `MVSRC` , `FILLCHR` , `FILLCNT` , `FILLADDR` , `HSLEN` , `HSADDR`) were used throughout `MOVE` / `CMOVE` / `CMOVE>` , `FILL` , `HOLDS` , and the single-cell multiply routine but never declared anywhere — a real, assembly-breaking gap.** Auditing every one of the 142 already-declared GLOBALS cells for actual use found zero dead cells to reclaim this time (unlike `SNEND` earlier), and the GLOBALS page was independently confirmed at exactly 256/256 bytes with no headroom. Since `MOVE` -family, `FILL` , `HOLDS` , and the multiply routine never call each other or run concurrently in this single-threaded interpreter, they now share physical storage: three new cells (`MVCNT` , `MVDST` , `MVSRC`) hold the real storage, and `FILLCNT` / `HSLEN` alias `MVCNT` , `FILLADDR` / `HSADDR` alias `MVDST` , and `MRESULT` / `FILLCHR` alias `MVSRC` , all via `EQU` . These three new cells could not fit in the full GLOBALS page, so they live at `$0100` , carved from the front of `USER0` (shrunk from 128 to 122 bytes, now starting at `$0106`) — a real, documented tradeoff: these three cells use ordinary extended addressing, not direct-page, including inside `CMOVEW` 's per-byte copy loop.

- **`JSR SPACE` (bare, undefined) — same class of bug as `CR` , fixed the same way.** `WORDS` called the bare, undefined `SPACE` instead of the actual routine label `SPACEW` (section 22). Fixed to `JSR SPACEW` . Checked `SPACES` for the same pattern (none found — `SPACESW` correctly calls `EMIT` directly) and swept the file for any other bare `SPACE` instruction reference (none remain; the two harmless bare "SPACE" occurrences left are a section-title comment and the dictionary header's `FCC "SPACE"` name string).

- **Three short branches (`BHS` , `BEQ` , `BNE`) overflowed the 8-bit short-branch range (±127 bytes) — a real assembler error, not a style issue.** `SUBCOPY` 's overflow check (`BHS SUBOVERFLOW` , ~87 source lines to target), `DUMPW` 's per-line loop test (`BEQ DUDONE` , ~76 lines), and `DUMPW` 's loop-back (`BNE DULINE` , ~76 lines) all spanned too much code for an 8-bit displacement. Fixed by converting each to its long-branch equivalent (`LBHS` / `LBEQ` / `LBNE`), which uses a 16-bit displacement (±32767 bytes) - functionally identical, just not range-limited.

- **Ten more short branches have a large (41-51 source line) span to their target and were not individually confirmed safe.** Found by a heuristic sweep (line-count as a rough proxy for byte distance, since no real assembler is available in this environment to get an exact count) after fixing the three confirmed overflows above, all of which spanned 76+ lines - these ten are meaningfully shorter but not verified within range: `QUERY` 's `QLOOP` (three branches, lines 579/582/589), `FIND` 's `NOTFOUND` / `FFLOOP` (lines 1210/1256), `ACCEPT` 's `ALOOP` / `ADONE` (lines 1501/1462), `UNESCAPE` 's `UEDONE` / `UELOOP` (lines 3888/3931), and `EMPTY` (line 1153). None of these were reported as failing and none have been changed; if a real assembler flags any of them, the fix is the same:

convert to the matching `LBxx` long-branch form.

- **`ORG BASECODE` was missing entirely — a fundamental placement bug, not a cosmetic gap.** `VECTORS` ($FFF0) and `INIT` ($FFC0) both had their own `ORG`, but `SECTION 3` (`IRQH`) — and every routine through `SECTION 26`, i.e. nearly the entire interpreter — had no `ORG` placing it in `BASECODE` at all. Without it, this code would have continued growing from wherever `INIT`'s WARM message left the location counter, inside `INIT`'s own 48-byte `$FFC0-$FFEF` budget, overflowing directly into `VECTORS` rather than landing in `BASECODE` ($E800-$FFBF) anywhere. Fixed by adding `ORG BASECODE` immediately before `SECTION 3` begins.
- **Superseded by a later, more precise count** (see the "Follow-up" entry below with the 7984-byte instruction-by- instruction result) - the 8660-byte figure here used a cruder heuristic and a 6080-byte budget that's since changed (`BASECODE` is now 8110 bytes nominal, after the `BASECODE` / `BASEDICT` 30-byte shift). Kept as history, not deleted, since it was a genuine finding at the time - just not the current best estimate. Original text: **Adding the missing ORG makes `BASECODE`'s byte budget concretely checkable for the first time — and a rough estimate suggests it may not fit.** This was previously flagged only as "unverified" (no real assembler has ever been run against this file); with a real `ORG` boundary now in place, a heuristic per-instruction byte count (inherent/direct-page/ indexed/immediate/extended opcode sizes, correctly distinguishing direct-page GLOBALS operands from extended ones) over every instruction from `SECTION 3` through `SECTION 26` estimated roughly 8660 bytes against a 6080-byte budget — about 2580 bytes over. The underlying question (does `BASECODE` actually fit, confirmed by a real assembler) is still genuinely open - see the later, more precise follow-up entry.
- **`USER0` and `USER1` (250 bytes of unallocated reserve space, never read or written by any code) were deleted, and the region from `MVSCRATCH` through `APPDICT` was made fully contiguous.** `SERBUF`, `INBUF`, `OUTBUF`, `TIBBUF`, `WORDBUF`, and `SIBUF` all shifted down to sit immediately after `MVSCRATCH` ($0106) with no gaps between any of them. This also closed a separate, pre-existing 11-byte gap between `WORDBUF` and `SIBUF` ($02F5-$02FF) that had nothing to do with `USER0` / `USER1` but was caught while verifying the region was genuinely contiguous end to end. `APPDICT` was extended downward to close the resulting gap too (new start $021B, was $0320) — its end ($6EFF) is unchanged, so this is a pure 261-byte gain in application dictionary space, not a resize of its budget. This was an interpretation of "contiguous," not something explicitly asked for beyond the 6 buffers themselves; flagged here in case a fixed `APPDICT` start was actually intended instead. Verified: zero duplicate symbols, zero remaining references to `USER0` / `USER1` anywhere in the code, and the full `GLOBALS` -> `MVSCRATCH` ->buffers-> `APPDICT` span checked programmatically for zero gaps.

- **APPVARS and APPDICT swapped positions - APPVARS now sits below APPDICT .** APPDICT moved from $021B to $031B (still ending at $6FFF , directly below APPCODE); APPVARS moved from $6F00 down to $021B (same 256-byte size, now sitting directly above the buffers instead of directly below APPCODE). Checked before making the change: only two code references exist for either symbol (COLD initializing VARHERE / DPHERE to each region's start) and neither depends on their relative order, so the swap was safe. Verified: zero duplicate symbols, the whole region from GLOBALS through APPCODE 's start remains contiguous with zero gaps, dictionary chain unaffected (219 entries, since this change doesn't touch it at all).

- **APPVARS grown from 256 to 8000 bytes, taking the space directly from APPDICT .** APPVARS now spans $021B-$215A (start address unchanged); APPDICT shrank by the same 7744 bytes to $215B-$6FFF (end unchanged, still directly below APPCODE), giving 20133 bytes of application dictionary space, down from 27877. Checked before making the change: still only two code references to either symbol (COLD 's VARHERE / DPHERE initialization), neither size-dependent. Verified: zero duplicate symbols, APPVARS size is exactly 8000 bytes, the whole region stays contiguous end to end, dictionary chain unaffected.

- **ACIA (the 256-byte memory-mapped I/O block) renamed to INOUT , and the actual 6850 ACIA chip's registers moved to INOUT+8 instead of the block's base.** The block still spans the same 256 bytes ($DF00-$DFFF); it's now modeled as a general I/O region with the ACIA chip occupying one small part of it (ACIACR / ACIASR = INOUT+8 = $DF08 , and the data register ACIADR at $DF09), leaving INOUT+0 .. INOUT+7 free for other memory-mapped devices sharing the block. Doing this rename surfaced a real, previously-hidden bug: COLDSTRT 's ACIA reset sequence used STA ACIA (the block's base) instead of STA ACIACR (the control register) in two places - this only ever worked because ACIA and ACIACR happened to be the same address in the old scheme. Once ACIACR moved to INOUT+8 , writing to the bare block base would have silently stopped resetting the ACIA at all. Fixed both to STA ACIACR . Verified: INOUT defined once, the bare ACIA symbol (at the time) no longer existed anywhere, and every other ACIA register access in the file already used the correctly-named register symbols rather than the bare block name.

- **ACIA reintroduced as an explicit base symbol (ACIA EQU INOUT+8), with ACIACR / ACIASR now defined relative to it (EQU ACIA) rather than directly to INOUT+8 .** The data register was briefly renamed to ACIRDR in the same pass, then reverted back to ACIADR immediately afterward once flagged as a typo - both code sites (IRQH 's receive and transmit paths) were updated each time the name changed. Verified: each of ACIA / ACIACR / ACIASR / ACIADR defined exactly once, resolving to the

expected values (`ACIA = INOUT+8` , `ACIACR` / `ACIASR = ACIA` , `ACIADR = ACIA+1`), zero remaining references to `ACIRDR` anywhere, zero duplicate symbols, dictionary chain unaffected.

- **`INITEND` / `INITSIZE` landmark constants added at the true end of the INIT block** (`INITEND EQU *` , `INITSIZE EQU *-COLDSTRT`), right after `WARMMSGL` . The request referenced a `COLDSTART` label, which doesn't exist in this file - used the actual existing label `COLDSTRT` instead of introducing a new undefined symbol.

- **`INITEND` 's comment now states the invariant explicitly ("value should match vector ORG") - and checking it confirms it currently holds.** The precise instruction-by-instruction byte count from when the `VECTORS` collision was first found (78 bytes exactly, `COLDSTRT` through `WARMMSGL`) gives `INITCODE start ($FFA2) + 78 = $FFF0` , exactly matching `VECTORS ' ORG` . Zero gap, zero overlap - unlike the still-open `INITCODE` / `BASECODE` overlap at the other end of this same region, which remains a real problem. Same caveat as always: this is a manual count, not a real assembler's output, so `INITSIZE` (computed automatically at assembly time) is the figure to trust once this file is actually assembled.

- **`BASECODESTRT` / `BASECODEEND` / `BASECODESIZE` and `BASEDICTSTRT` / `BASEDICTEND` / `BASEDICTSIZE` landmark constants added, matching the `INITEND` / `INITSIZE` pattern - and checking the two stated invariants gives one clean confirmation and one confirmation of an already-known problem.** `BASEDICTEND` (`$D85D` + the exact 1973-byte dictionary content = `$E012`) matches `ORG BASECODE` (`$E012`) precisely - this boundary is genuinely correct, not just close. `BASECODEEND` , using the same rough heuristic estimate flagged much earlier (~8660 bytes, never confirmed with a real assembler) starting from `BASECODESTRT` (`$E012`), lands around `$101E6` - not only failing to match `ORG INITCODE` (`$FFA2`) as the comment expects, but overflowing past `$FFFF` entirely on that estimate. The exact amount of room actually available between `BASECODESTRT` and `INITCODE` ($FFA2 - $E012 = 8080 bytes) is itself less than the ~8660-byte estimate, meaning even filling every available byte up to `INITCODE` with zero gap would still likely fall short. This is the same underlying problem already tracked elsewhere (`BASECODE` possibly not fitting its budget, and separately the `INITCODE` / `BASECODE` overlap), now independently reachable via these new landmarks once the file is actually assembled, rather than a new, different issue.

- **Follow-up: a real instruction-by-instruction count (not the rough heuristic above) puts `BASECODE` 's actual size at 7984 bytes (`$1F30`), 96 bytes (1.2%) short of a target assembler- listing value of `$1F90` (8080 bytes) given for comparison.** Built a mnemonic-and-addressing-mode-aware counter distinguishing

inherent/immediate/direct/extended/indexed forms, correct prefix requirements (`$10` / `$11` for `LDY` / `STY` / `LDS` / `STS` / `CMPD` / `CMPY` / `CMPU` / `CMPS`), and true direct-page symbols (only `$0000` - `$00FF` , matching `DP`) - a meaningfully more rigorous pass than the original ~8660-byte estimate. Every one of the 3751 instructions in the region was classified (zero "unrecognized" remaining after fixing early misses like `LSLB` / `LSLA` as `ASLB` / `ASLA` synonyms). Checked for likely sources of the remaining 96-byte gap before accepting it: zero indexed operands use an offset outside the 5-bit range that would need extra encoding bytes (all 125 numeric-offset operands found are small, e.g. `1,X` / `-1,Y`), and the two apparent "indirect `[ ]` " matches were a false positive (a section-title comment, not real indirect addressing). The target value `$1F90` is notable in its own right: `BASECODE + $1F90 = $FFA2` exactly, meaning if the real assembled size actually matched it, `BASECODE` and `INITCODE` would be perfectly contiguous with zero gap and zero overlap - which would also resolve the separate, still-open `INITCODE` / `BASECODE` overlap noted elsewhere. My count doesn't confirm that, though it's close (1.2% off). This is still not a real assembler's output - the standing caveat throughout this file - and the gap could come from encoding edge cases a manual model can approximate but not guarantee against.

- **`ROMSTRT` / `ROMEND` added as the outer ROM boundary (`$C100` / `VECTORS-1`), and `INITCODE` / `BASECODE` / `BASEDICT` verified contained within it - all three pass.** `ROMEND` was initially defined as `$10000` ("one past `VECTORS` 's end," a symbolic value that doesn't fit as a real 16-bit address), then corrected to `VECTORS-1` (`$FFEF` , one before `VECTORS` 's start) - a genuine 16-bit address usable directly in comparisons or as a memory operand, not just symbolic arithmetic. Re-checked after the correction rather than assumed still valid: `INITCODE` ($FFA2-$FFEF), `BASECODE` ($E012-$FFBF), and `BASEDICT` ($D85D-$E011) are all still genuinely within `ROMSTRT..ROMEND` - `INITCODE` 's own end now lands exactly on the new boundary, a tighter and more meaningful check than before, not a violation. This containment check passes cleanly, independent of the other open problems below.

- **`ROMSTRT` / `ROMEND` renamed to `USROMSTRT` / `USROMEND` , each with a short "Usable ROM start"/"Usable ROM end" comment.** Cosmetic, not functional - neither symbol was referenced by any code, only by nearby comments (three sites, all updated: the pair's own definitions and `INOUT` 's cross-reference). Verified: zero remaining bare `ROMSTRT` / `ROMEND` references anywhere, zero duplicate symbols, dictionary chain unaffected.

- **`VECTOREND` / `VECTORSIZE` landmark constants added at the true end of the vector table, matching the `INITEND` / `INITSIZE` pattern - and checking the stated invariant confirms it exactly, not approximately.** Unlike `INITEND` / `BASECODEEND` (which needed a manual, approximate instruction-by-instruction byte count since real 6809

instructions have variable-length encoding), the vector table is pure `FDB` data with a fixed, unambiguous 2-byte width per entry - counting the 8 entries (`VRESV` through `VRESET`) gives exactly 16 bytes, matching the comment's stated `$10` expectation precisely. Verified: zero duplicate symbols, dictionary chain unaffected.

- **`BASECODESTRT` removed as requested - but `BASECODESIZE` depended on it, so removing it alone would have left a genuine dangling undefined-symbol reference, the same bug class found and fixed several times earlier in this file.** Confirmed first that `BASECODESTRT` was numerically identical to `BASECODE` itself (only comments sit between `ORG BASECODE` and where `BASECODESTRT` was defined, zero emitted bytes), then fixed `BASECODESIZE` to read `BASECODEEND-BASECODE` directly instead - matching the same pattern just applied to `INITSIZE` / `INITCODE` , not a new approach invented for this case. Verified: zero remaining `BASECODESTRT` references anywhere, `BASECODESIZE` resolves cleanly, zero duplicate symbols, dictionary chain unaffected.

- **`BASEDICTSTRT` removed the same way, for the same reason - `BASEDICTSIZE` depended on it too.** Same fix pattern as `BASECODESTRT` immediately above, applied to its counterpart: confirmed `BASEDICTSTRT` was numerically identical to `BASEDICT` (only `ORG BASEDICT` sits before it, zero emitted bytes), removed it, and repointed `BASEDICTSIZE` to `BASEDICTEND-BASEDICT` directly. Verified: zero remaining `BASEDICTSTRT` references anywhere, `BASEDICTSIZE` resolves cleanly, zero duplicate symbols, dictionary chain unaffected.

- **`INOUT` moved from `$DF00` to `$C000` (with `INOUTEND EQU INOUT+$FF` added immediately after), and every ACIA-related address verified between the two - also passes.** `ACIA` / `ACIACR` / `ACIASR` (`$C008`) and `ACIADR` (`$C009`) all fall within `INOUT` - `INOUTEND` (`$C000` - `$C0FF`), checked numerically. Confirmed there is no other memory-mapped I/O device anywhere in this file besides the ACIA family - nothing else needed checking. This move has a real, positive side effect worth naming clearly: it resolves the INOUT portion of the three-way collision flagged when `BASEDICT` moved to `$D85D` two turns ago (`INOUT` no longer overlaps `BASEDICT` , since `$C0FF < $D85D`). The `DSTACK` and `RSTACK` portions of that same collision were untouched by this specific change, but have since been resolved separately - see below.

- **`INOUT` 's new collision with `APPCODE` is resolved, and so is the rest of the original `BASEDICT` collision - `RSTACK` , `DSTACK` , `CODETOP` , `APPCODE` , and `APPDICT` all moved down `$2000` .** `RSTACK` ($BC00-$BEFF) and `DSTACK` ($B800-$BBFF) no longer overlap `BASEDICT` at all - the three-way collision first found when `BASEDICT` moved to `$D85D` is now fully resolved (`INOUT` resolved it two turns ago, this resolves the remaining two thirds). `APPCODE` 's move ($5000-$B7FF) also separately resolves its overlap with `INOUT` from the previous entry, as a side effect of the same shift, not a second fix. Re-verified

with a full pairwise sweep after the move, not assumed: the only overlap remaining from the four found last turn is the original, unrelated `INITCODE` / `BASECODE` one (30 B) - `INITCODE` / `BASECODE` / `BASEDICT` / `RSTACK` / `DSTACK` / `INOUT` / `APPCODE` are all now mutually clean.

- **`APPDICT` 's collision with `APPVARS` is now resolved - `APPVARSEND` changed from a static `APPVARS+8000` to `APPDICT-1` , making it self-derive from wherever `APPDICT` actually sits instead of a fixed size.** `APPVARSEND` is now `$1FFF` (was `$215B`), giving `APPVARS` 7653 bytes of actual usable space (down from the originally-intended 8000 - the 347-byte reduction exactly matches the overlap this closes). `APPVARS` 's own `EQU` still starts at `$021B` and its comment still cites the historical "8000 bytes" intent, now explicitly marked as the original intent rather than the current usable size. Functionally, `APPVARS` 's enforced range (`$021B-$1FFF`) no longer reaches `APPDICT` (`$2000-$6EA4`) at all - checked numerically, not assumed. This also makes the boundary self-correcting: if `APPDICT` moves again, `APPVARSEND` moves with it automatically, rather than needing another manual fix like the last several turns' worth of address changes required. `VUNUSEDW` 's own code is unchanged - it already computed against `APPVARSEND` (see below), so this fix took effect purely by changing what that symbol means. Verified: zero duplicate symbols, dictionary chain unaffected.
- **`DSTACK` moved up `$200` (to `$BCFF`), resolving both the `CODETOP` / `DSTACK` mismatch and the `RSTACK` / `DSTACK` gap from last turn in one move.** `DSTACK` 's new occupied bottom (`$B900` , from its `$BCFF` top and 1024-byte size) now matches `CODETOP` (`$B900`) exactly - `APPCODE` 's nominal ceiling no longer overstates safe growth room, and the 512-byte overlap that created between `APPCODE` 's nominal range and `DSTACK` 's true range is gone. As a bonus, not something separately requested: `DSTACK` 's new top (`$BCFF`) is now exactly contiguous with `RSTACK` 's bottom (`$BD00`), closing the 512-byte gap that had opened between them too. Verified with a full pairwise sweep after the move: exactly two overlaps remain in the current memory map - the still-open `APPDICT` / `APPVARS` one (347 B) and the original, unrelated `INITCODE` / `BASECODE` overlap (30 B) - down from three last turn.
- **The VECTORS collision found by verifying `INITEND` is now resolved - `INIT` 's ORG moved from `$FFC0` to `$FFA0` .** Widened `INIT` from 48 to 80 bytes ($FFA0-$FFEF), comfortably covering the ~78 bytes `COLDSTRT` + `WARM` + `WARMMSG` actually needs (2 bytes of margin) - still an estimate from manual byte-counting, not a real assembler's output. `INIT` 's own `ORG` was also converted from a literal `$FFC0` to symbolic `ORG INIT` , matching `BASECODE` / `BASEDICT` 's convention, so the `EQU` and the actual placement can no longer drift apart the way `BASECODE` 's did before that was caught. One real, unresolved interaction worth naming: `INIT` 's new start ($FFA0) is 32 bytes below the old documented `BASECODE` end ($FFBF), tightening the still-open `BASECODE` overflow

risk below by that much further - not a new problem in kind, since that risk was already estimated at roughly 2580 bytes over budget, dwarfing this additional 32-byte reduction, but worth tracking alongside it rather than treated as independent.

- **RESOLVED (since this was written): the `INITCODE` / `BASECODE` overlap described below (30 bytes at the time) was fully closed several turns later** by shifting `BASECODE` and `BASEDICT` both down exactly 30 bytes - `BASECODE` 's nominal end now lands exactly one byte below `INITCODE` 's start, zero gap, zero overlap, confirmed by a full pairwise sweep of the entire memory map. Kept as history below, not deleted. Original text: **`INIT` moved again, from `$FFA0` to `$FFA2` (request contained a `&FFA2` typo - used the correct `$` hex prefix instead).** `INIT` is now exactly 78 bytes ($FFA2-$FFEF), matching the `COLDSTRT` + `WARM` + `WARMMSG` byte-count estimate with zero margin (was 80 bytes, with 2 bytes of slack). This is a genuinely tighter fit than before, worth naming as a real tradeoff: any inaccuracy in the manual byte count, or any future addition to `COLDSTRT` / `WARM` , now has zero room to absorb. The overlap with `BASECODE` noted above is **not resolved by this change, only reduced** - from 32 bytes to 30 bytes ($FFA2-$FFBF). This was a real, pre-existing problem before this turn touched anything (`$FFA0` already overlapped `BASECODE` 's declared end by 32 bytes), not something newly introduced. Not fixed here, in either direction (shrinking `BASECODE` further or moving `INIT` above it instead of below): both are bigger architectural decisions than "change one EQU," and a real assembler run would settle the actual `BASECODE` byte count this depends on.

- **`INIT` renamed to `INITCODE`** (`EQU` and its `ORG` reference, both in the memory-map constants). The section-title comment ("SECTION 2: INIT CODE") and one historical narrative comment describing a past bug scenario (with the byte figures true at that time, not today) were deliberately left referring to "INIT" as plain English/history, not the symbol - renaming a symbol doesn't obligate rewriting prose that correctly describes what was true before the rename existed. Doing this rename surfaced that the documentation's "ROM Size Required" section (Section 2) had gone stale independent of the rename itself: it still asserted the four ROM regions were contiguous at exactly 8192 bytes, which stopped being true as soon as `INIT` moved to `$FFA2` last turn and started overlapping `BASECODE` . Rewritten to state the overlap plainly instead of the now-false claim. Verified: `INITCODE` defined exactly once, zero duplicate symbols, dictionary chain unaffected.

- **`BASEDICT` and `BASECODE` addresses applied exactly as requested, and the documented ROM requirement widened to a 16K part - but this surfaced a severe, unresolved collision, not a minor tradeoff.** `BASEDICT` moved from `$E000` to `$D85D` - verified before making the change that `$E012 - $D85D = 1973` bytes exactly matches SECTION 27's real, measured dictionary content, a genuine zero-padding exact fit (was

2048 bytes, 75 bytes of slack). `BASECODE` moved from `$E800` down to `$E012` to match; its own upper bound was not given and stayed at `$FFBF`, growing it from 6080 to 8110 bytes.

- **RESOLVED (since this was written): the three-way collision described below is fully closed.** `INOUT` moved first (resolving the `INOUT` third), then `RSTACK` / `DSTACK` / `CODETOP` / `APPCODE` / `APPDICT` were all given new addresses (resolving `DSTACK` / `RSTACK`), and `BASEDICT` itself later moved again to `$D83F` (from `$D85D`, shifting down 30 bytes alongside `BASECODE`) as part of closing the separate `INITCODE` / `BASECODE` overlap. A full pairwise sweep of the current memory map (`VECTORS` / `INITCODE` / `BASECODE` / `BASEDICT` / `INOUT` / `RSTACK` / `DSTACK` / `APPCODE` / `APPDICT` / `APPVARS` /all buffer regions/ `GLOBALS`) confirms **zero overlaps anywhere**, re-checked as part of this same update. Kept as history below, not deleted. Original text: **`BASEDICT` 's new range ($D85D-$E011) entirely overlaps three other live regions: `DSTACK` (931 of its 1024 bytes, `$D85D-$DBFF`), `RSTACK` (all 768 bytes, `$DC00-$DEFF`), and `INOUT` (all 256 bytes, `$DF00-$DFFF`).** Found by checking the new address against every other region's boundaries before updating the documentation - not caught until then. This is a fundamental, system-breaking conflict, not a byte-budget tightness issue like the `INITCODE` / `BASECODE` overlap noted elsewhere: as configured, the ROM dictionary, the data stack, the return stack, and the ACIA's registers would all be mapped to the same physical addresses simultaneously, which cannot work on real hardware without bank switching (which this system does not have). The requested `EQU` values were applied exactly as given, since that's what was asked; resolving the collision itself was not attempted here, since it requires a decision this response can't make alone - moving `DSTACK` / `RSTACK` / `INOUT` elsewhere, choosing a smaller `BASEDICT` / `BASECODE` split that doesn't reach down this far, or reconsidering whether `$D85D` was the address actually intended. The documented ROM part was still widened from 8K to 16K per the request (real total usage of `BASEDICT` + `BASECODE` + `INITCODE` + `VECTORS` alone, ignoring the collision, is about 10.15K, leaving roughly 6.2K of spare capacity within a 16K×8 EPROM), but that figure is secondary to the collision above. Verified: zero duplicate symbols, dictionary chain unaffected (219 entries, since this doesn't touch it).
- **The `INITCODE` / `BASECODE` overlap - open since it was first found, mentioned in at least five separate entries above across several turns - is finally resolved.** **`BASECODE` and `BASEDICT` both shifted down exactly 30 bytes** (`BASECODE`: `$E012` -> `$DFF4`; `BASEDICT`: `$D85D` -> `$D83F`), chosen precisely: 30 bytes is exactly the overlap amount, so `BASECODE` 's nominal end (`$FFA1`, unchanged 8110-byte budget) now lands exactly one byte below `INITCODE` 's start (`$FFA2`) - zero gap, zero overlap. `BASEDICT` shifted the same amount to stay perfectly contiguous with `BASECODE` 's new start,

preserving its own exact, zero-padding fit. Verified with a full pairwise sweep across every region in the memory map, not just the two that moved: **zero overlaps anywhere** - the first time that's been true since this whole sequence of address changes began. `USROMSTRT` / `USROMEND` containment re-checked and still passes for all three ROM regions. Also cleaned up the accumulated resolved-issue narrative in the top-of-file note (previously ~40 lines chronicling the `BASEDICT` / `DSTACK` / `RSTACK` / `INOUT` / `CODETOP` / `APPDICT` saga turn by turn, including one claim - the `APPDICT` / `APPVARS` overlap being open - that was already stale before this edit) down to a short, current-state summary, matching the same cleanup already applied to the documentation.

- **`forth6809.asm` 's four ORG'd blocks physically reordered to match ascending memory address, low to high: `BASEDICT` , `BASECODE` , `INITCODE` , `VECTORS`** (previously `VECTORS` , `INITCODE` , `BASECODE` , `BASEDICT` - roughly the reverse). Pure text reordering only - `ORG` directives set the actual assembled address independent of where in the file each block appears, so this has zero effect on the assembled output; verified rather than assumed by comparing the sorted multiset of every line in the file before and after, which matched exactly (nothing added, removed, or altered - only position changed). This was also done in a freshly-reset environment (the working directory from earlier turns was gone; recovered by copying the last-delivered `forth6809.asm` back from the persistent output location and confirming its hash matched what was last verified). The dictionary chain-walk check initially appeared to fail after reordering (1 of 219 entries reached) - traced to a bug in the verification script itself, not the file: an arbitrary 300-character cap on how much of each header's body text it scanned cut off `ABORTHDR` 's second `FDB` field, since its comment runs several lines longer than most headers. Fixed the script to scan each header's full body instead of a fixed character count, re-ran, and confirmed all 219 entries reachable end to end. Also confirmed zero duplicate symbols and all four region `EQU` s still resolve to their correct, unchanged addresses.

- **`ROMSTRT` / `ROM:` / `FILL` padding block's 256-byte gap is now closed - the `ORG` target changed from `ROMSTRT` to `USROMSTRT` , and `ROMSTRT` itself moved to the top of the memory-map constants, alongside `USROMSTRT` / `USROMEND` rather than sitting alone just before its point of use.** The gap flagged last turn wasn't a formula problem - `BASEDICT- USROMSTRT` (5951 bytes) was already the right byte count, it just needed to start from `USROMSTRT` ($C100), not `ROMSTRT` ($C000). With `ORG USROMSTRT` , the fill now covers `$C100` - `$D83E` exactly, landing with zero gap against `BASEDICT` 's real start (`$D83F`). `ROMSTRT` remains a distinct, real symbol (`$C000` , the true physical EPROM start, still meaningfully different from `USROMSTRT`), now serving purely as a documented reference constant rather than an `ORG` target. `FILL` 's status as an unconfirmed LWASM directive remains open - that wasn't addressed this turn - see the note left in place at the actual `FILL` line. Also fixed a second stale comment found while making these

changes: “BASECODE is $E800” (on the `ORG BASECODE` line, left over from several address changes ago) corrected to $DFF4; “BASEDICT is $E000” was checked and found already correct from a prior turn, needing no change. Verified: zero duplicate symbols, `ROMSTRT` still defined exactly once at its new location, zero remaining `ORG ROMSTRT` anywhere, dictionary chain still 219 entries intact. Also worth noting: this turn’s work began by discovering the local working copy had silently diverged from the actually-delivered file (showing `ORG USROMSTRT` when the delivered file actually had `ORG ROMSTRT`) - resolved by re-syncing the working copy from the persistent, hash-confirmed output file before making any changes, rather than editing from an unverified state.

- **A second `FILL` padding block added, this time between `BASECODE` ’s real content and `INITCODE` ’s start - same intent and same open verification question as the `ROM:` block two turns ago.** `BASEND:` sits right where `BASECODE` ’s actual assembled content ends (the same point `BASECODEEND` marks, immediately before `SECTION 2` begins, now that the file’s physical section order has `BASECODE` directly followed by `INITCODE`); `FILL $FF,INITCODE-BASEND` covers whatever gap exists between there and `INITCODE` ’s start. Genuinely self-correcting, not a hardcoded guess: because `BASEND` is computed from wherever real content actually ends rather than an estimated figure, this fill’s size will automatically be correct once a real assembler runs, regardless of the precision of any manual byte count. Using the precise instruction-by- instruction count from several turns ago (recomputed fresh here to confirm it’s still 7984 bytes, unchanged since nothing in the intervening turns touched `BASECODE` ’s actual content), this would cover 126 bytes (`$FF24` - `$FFA1`) - matching the slack between real content and the nominal 8110-byte budget exactly, as expected. `FILL` ’s status as an LWASM directive remains unconfirmed (see the `ROM:` block’s own note) - not re-verified here, since the question is identical for both uses. Verified: zero duplicate symbols, `BASEND` defined exactly once, dictionary chain still 219 entries intact.
- **Both padding blocks repositioned, and the `ROM:` block’s explanatory comment deleted entirely.** `ORG USROMSTRT` / `ROM:` / `FILL` moved from just before `ORG BASEDICT` to just before `SECTION 27` ’s comment header instead (now sitting directly after `GLOBALS_USED`); `BASEND:` / `FILL` moved from just before `ORG INITCODE` to just before `SECTION 2` ’s comment header (directly after `BASECODESIZE`). The multi-line “UNVERIFIED: FILL is not a directive…” comment on the `ROM:` block is gone - the underlying uncertainty (whether `FILL` is a real, supported LWASM directive) is unchanged and still real, it’s just no longer written down at that specific spot; see the historical entries above for the actual finding. Pure repositioning and deletion, nothing recomputed or changed in substance. Verified: zero duplicate symbols, `ROM:` / `BASEND:` each still defined exactly once, both blocks confirmed (by text position, not assumption)

to sit before their respective section's comment header rather than after, zero remaining "UNVERIFIED" text anywhere, dictionary chain still 219 entries intact.

- **`MVSCRATCH` (the `ORG $0100` block, `MVCNT` / `MVDST` / `MVSRC` and their aliases through `FILLCHR`) moved from right after `GLOBALS EQU $0000` to right after `GLOBALS_USED EQU 256` instead - continuing the same file-order-matches-memory-order cleanup as the `VECTORS` / `INITCODE` / `BASECODE` / `BASEDICT` reordering several turns back.** Moved as one unit: the full explanatory comment block plus all nine lines of actual code, ending exactly at `FILLCHR EQU MVSRC` as specified - `SP0` / `RP0` (unrelated stack- pointer constants that happened to sit right after `MVSCRATCH`) stay in their original position, now following `GLOBALS` directly. Verified with the strongest available check: the sorted multiset of every line in the file, compared against the version delivered immediately before this edit, matched exactly - confirming nothing was added, removed, or altered, only repositioned. Also confirmed the new order is correct (`GLOBALS_USED` -> `MVSCRATCH`'s `ORG $0100` -> `ORG USROMSTRT`, matching ascending memory address: `$0000` - `$00FF` globals, `$0100` - `$0105` MVSCRATCH, `$C100` + ROM), zero duplicate symbols, every `MVSCRATCH`-related constant still defined exactly once, dictionary chain still 219 entries intact.

- **Conditional assembly added for serial I/O: interrupt-driven (original) and polling (new) implementations now coexist, switched by a single flag, `SERIALPOLL EQU 1` (polling is the default/active mode).** Verified LWASM's actual conditional- assembly directives against the real manual before using them (`IFEQ` / `IF` / `IFNE` / `IFDEF` / `IFNDEF` / `ELSE` / `ENDC` are documented) rather than guessing, unlike the still-unresolved `FILL` question elsewhere. Three conditional blocks, all gated by the same flag: (1) `INFILL` / `RTSCHECKHI` / `RTSCHECKLO` / `IRQH` (the full interrupt-driven receive/transmit/RTS-handshaking logic) - only assembled when `SERIALPOLL=0`; when `SERIALPOLL=1`, `IRQH` becomes a bare `RTI` stub, matching the other genuinely unused vectors (`NMIH` / `FIRQH` / `SWI2H` / `SWI3H`), since ACIA interrupts are never enabled in polling mode and the vector table unconditionally references `IRQH` by name either way. (2) `KEY` / `KEYQ` / `EMIT` - both versions compile to the same three label names, so the dictionary headers (`H_KEY` / `H_KEYQ` / `H_EMIT`) never need to change regardless of which mode is active. The new polling versions block (`KEY`, `EMIT`) or check once (`KEYQ`) directly against `ACIASR`'s `SR_RDRF` / `SR_TDRE` bits - no ring buffers, no RTS/CTS, fully synchronous. (3) `COLDSTRT`'s ACIA mode-select, now choosing between `CR_RXON` (RX interrupt enabled) and a new `CR_POLL` (`$15` - `CR_RXON` with only the RX-interrupt-enable bit cleared; RTS held permanently low, since polling has no ring buffer to overflow and needs no flow control). Checked before writing any of this that no other code references `RTSSTATE` / `INFILL` / `RTSCHECKHI` / `RTSCHECKLO` / `CR_RTSHI` / `CR_RXTX` outside what's already being conditionally wrapped - confirmed clean, nothing left

dangling. Verification required a different approach than the usual duplicate-symbol check, since `KEY` / `KEYQ` / `EMIT` / `IRQH` legitimately appear twice in the raw source now (once per branch) - a naive check would false-positive on that. Instead, simulated what LWASM would actually assemble for each value of `SERIALPOLL` (stripping whichever branch is inactive) and ran the real structural checks against each simulated result separately: both branches independently show zero duplicate symbols, `KEY` / `KEYQ` / `EMIT` / `IRQH` each resolving to exactly one definition, and the dictionary chain fully intact (219 entries) in both cases. Also confirmed the three `IFEQ` / `ELSE` / `ENDC` blocks are balanced (3/3/3) in the raw file.

- **SUPERSEDED by a later entry below: moving `ABORTHDR` / `QUITHDR` into `BASEDICT` reintroduced a 19-byte `BASECODE` / `BASEDICT` overlap after this was written** - "zero overlaps anywhere" was accurate at the time, not any more. Kept as history, not deleted. Original text: **Current memory map state, re-verified fresh as of this update: zero overlaps anywhere.** A full pairwise sweep across all 18 regions (`VECTORS` , `INITCODE` , `BASECODE` , `BASEDICT` , `INOUT` , `RSTACK` , `DSTACK` , `APPCODE` , `APPDICT` , `APPVARS` , `SIBUF` , `WORDBUF` , `TIBBUF` , `OUTBUF` , `INBUF` , `SERBUF` idx, `MVSCRATCH` , `GLOBALS`) found no collisions - every gap/overlap that was ever open (the `INITCODE` / `BASECODE` 30-byte overlap, the `BASEDICT` / `DSTACK` / `RSTACK` / `INOUT` three-way collision, the `CODETOP` / `DSTACK` mismatch, the `APPDICT` / `APPVARS` overlap) has been resolved by earlier turns and stays resolved now, after the `SERIALPOLL` conditional-assembly addition and both `FILL` padding blocks, neither of which touch region boundaries. All three ROM-resident regions (`INITCODE` / `BASECODE` / `BASEDICT`) remain genuinely contained within `USROMSTRT..USROMEND` . What remains genuinely open, not a gap/overlap question: whether `BASECODE` 's real assembled size actually fits its 8110-byte nominal budget (the precise manual count is 7984 bytes, still not confirmed by a real assembler - see the earlier follow-up entry), and `FILL` 's status as an actual LWASM directive.

- **`ABORTHDR` / `QUITHDR` / `BASELATEST` moved from their own separate section into `BASEDICT` proper, at the end of the dictionary entries (right after `DICTTOP`), and renamed to `H_ABORT` / `H_QUIT` to match this file's `H_` naming convention.** This wasn't purely cosmetic: these two headers were previously sitting physically inside `BASECODE` 's address range (Section 26 sits before `ORG BASEDICT` in the file), even though they're header *data*, not code - meaning their bytes were being counted against `BASECODE` 's budget instead of `BASEDICT` 's. Moving them into the actual `ORG BASEDICT` ... `BASEDICTEND` block fixes that miscounting, at the cost of a new, real consequence: `BASEDICT` 's real size grew from 1973 to 1992 bytes (the 19 bytes `H_ABORT` and `H_QUIT` actually occupy), but `BASECODE` 's start (`$DFF4`) is a fixed address that doesn't move just because `BASEDICT` 's real content grew - reintroducing a genuine 19-byte overlap

between them (`$DFF4-$E006`), the same class of problem as the original `INITCODE` / `BASECODE` overlap resolved several turns ago. A full pairwise sweep of the entire memory map confirms this is the *only* new overlap - nothing else is affected. Not fixed here, since it needs a decision (shift `BASECODE` by 19 bytes to match, the same kind of fix used last time; or something else) rather than a mechanical follow-up. `TRUEBODY` / `FALSEBODY` (real code, correctly retitled) stayed in `BASECODE` where they belong - only the two header structures moved. Verified: zero duplicate symbols (checked against both `SERIALPOLL` branches via the same simulation-based method established for that feature), dictionary chain still 219 entries, now correctly walked starting from `H_QUIT` rather than `QUITHDR` , ending cleanly at `0` in both configurations. Also fixed two separately-stale items caught while making this edit: the Section 27 header comment still cited `BASEDICT` as `$D85D-$E011` (from before the 30-byte shift several turns back) and the top-of-file summary's "zero overlaps anywhere" claim, both now updated to reflect this change and its real consequence.

- **`BASECODE` shifted up 19 bytes (`$DFF4 -> $E007`) to close the `BASECODE` / `BASEDICT` overlap from last turn - but the overlap wasn't eliminated, it was relocated.** `BASECODE` 's new start does land exactly one byte above `BASEDICT` 's real end (`$E006`), closing that specific 19-byte overlap precisely. But `BASECODE` 's nominal size (8110 bytes) didn't change, so its nominal end shifted by the same 19 bytes too - from `$FFA1` to `$FFB4` , which is now 19 bytes *into* `INITCODE` 's start (`$FFA2`). A full pairwise sweep of the entire memory map confirms exactly one overlap remains, same total byte count (19), just at a different boundary (`INITCODE` / `BASECODE` instead of `BASECODE` / `BASEDICT`). This was checked and reported precisely, not assumed away or glossed over: shifting only `BASECODE` 's *start* while its budget stays fixed cannot reduce total overlap when that budget was already calibrated (in an earlier turn) to exactly reach `INITCODE` 's start from the *old* position - moving the start without shrinking the budget just pushes the same problem to the other end. Actually eliminating both overlaps at once would need `BASECODE` 's nominal size reduced by 19 bytes (to 8091), not just its start address moved; not done here, since that's a different change than what was asked. Verified: zero duplicate symbols in both `SERIALPOLL` branches (same simulation method as before), dictionary chain still 219 entries in both configurations. Also fixed a real arithmetic error caught before delivery: an initial draft of this comment miscalculated the new `INITCODE` overlap as 7 bytes at `$FFAE` - corrected to the actual 19 bytes at `$FFB4` before this was ever shipped.

Structural duplication (identified, some resolved, some not)

- `:` / `CREATE` / `VARIABLE` 's header-building — resolved via `HEADER` (section 7).

- `COMMA` / `CODECOMMA` / `CCOMMA` / `VCOMMA` / `VCCOMMA` / `CCOMMA1` — resolved via `APPENDCELL` / `APPENDBYTE` (section 6).
- `<#` / `#>` recomputing PAD's address inline — resolved, both now call `PADW` (section 20 / section 6).
- No further known duplication, but the codebase was never given a full pass specifically hunting for more.

Real, load-bearing gaps

- **Software (XON/XOFF) handshaking is still not implemented.** Now that hardware RTS/CTS handshaking is in place, this is the one remaining flow-control gap: this system still never transmits XON/XOFF and would not recognize either byte as a control signal if the remote device sent them — an incoming `$11` / `$13` is just queued as an ordinary character. Only relevant for links with no RTS/CTS wiring (plain 3-wire serial); would need a byte- interception layer between the input ring and `KEY` 's caller.
- **No hard boundary checks** anywhere `DPHERE` / `CODEHERE` / `VARHERE` grow toward each other or toward the stacks. `UNUSED` and `VUNUSED` both correctly *report* the distance to their boundary now (`CODETOP` and the newly-added `APPVARSEND` respectively), but nothing enforces either — a runaway compile can silently corrupt an adjacent region.
- **`VUNUSEDW` (the `VUNUSED` word) computed a meaningless number, not "how much `APPVARS` space remains" - a real, distinct bug, not just the absence of a check. Fixed.** `LDD #APPCODE / SUBD VARHERE` computed `APPCODE - VARHERE` (roughly `$7000` minus wherever `VARHERE` currently sits within `APPVARS`), which had nothing to do with `APPVARS` 's own boundary - looked like a copy-paste slip from `UNUSEDW` immediately above it (`LDD #CODETOP / SUBD CODEHERE` , correct for `CODEHERE` 's own boundary). Surfaced directly by a question about how `APPVARS` 's size is actually set: it wasn't - `APPVARS EQU $021B` defined only its start; there was no end/size constant anywhere. Fixed by adding `APPVARSEND EQU APPVARS+8000` (an exclusive upper bound, `$215B` , matching `CODETOP` 's own convention for `APPCODE`) and changing `VUNUSEDW` to `LDD #APPVARSEND / SUBD VARHERE` . Verified: at cold start (`VARHERE` = `APPVARS`), this computes exactly 8000, matching `APPVARS` 's documented size precisely; zero duplicate symbols; dictionary chain unaffected. `APPVARSEND` initially fell within `APPDICT` 's current range - expected at the time, the same, already-tracked `APPDICT` / `APPVARS` overlap, not something this fix caused. Since resolved by redefining `APPVARSEND` as `APPDICT-1` - see above.
- **`ENVTABLE` is incomplete.** Missing entries: `/HOLD` , `/PAD` (this build never fixed a capacity for either — filling them in would mean deciding a real bound first, not just

picking a number), `MAX-D`, `MAX-UD` (need the dispatcher extended to push *two* cells for double-cell answers — current `ENVQUERY` only handles one), `WORDLISTS` / `FLOORED` (need the dispatcher to be able to report a recognized-but-false answer, distinct from "unrecognized name" — no such path exists yet).

- **`CATCH`-wrapped `QUIT` / `INTERPRET` rollback is scoped to one input line only.** A colon definition spanning multiple lines that fails partway through a later line only rolls back that line's contribution, not the whole definition back to `:`. A complete fix needs the region-pointer snapshot taken once at `:` and held until `;` or an error, not refreshed every `QLOOP` pass.
- **`ABORTHDR`'s `LINK` field is a placeholder** `0` in the consolidated/split source — must be set to the real prior `LATEST` value once final ROM layout/assembly order is fixed.

Never resolved after being explicitly raised

- **`J` doesn't generalize past one level of loop nesting.** No `J2` /deeper equivalent exists. A triple-nested loop's innermost body has no built-in way to reach the outermost index without manually replicating `J`'s offset arithmetic one frame further.
- **`LEAVE`'s correctness depends on always being textually inside the loop it affects** — never verified against a `LEAVE` called from a separately-defined word invoked from within a loop (which would read/set the wrong stack frame, or crash).
- **`WORD`'s 31-character cap is now inconsistent within the system.** `PARSE` / `PARSE-NAME` were deliberately rewritten to not share it, but `WORD` itself — and everything still built on it (`CHAR`, `[CHAR]`, header names, `FIND`) — still has it.
- **The optional Search-Order word set is not implemented.** Every word resolves through one unified dictionary chain rooted at `LATEST`; there is no multi-wordlist support (`WORDLIST`, `GET-ORDER` / `SET-ORDER`, `ALSO` / `ONLY`, etc.). This was a foundational decision fixed since `COLDSTRT`'s first version, not an oversight — now documented explicitly (ClaudeForth documentation, Section 4.5) along with what adding it later would actually require: restructuring `FIND` into a loop over an active search order, changing `HEADER` to link into a selectable "current" wordlist instead of unconditionally into `LATEST`, and fixing the few words (`RECURSE`, `;`'s unsmudge step, `WORDS`) that read `LATEST` directly today.

Open design questions (not defects — deliberate unresolved choices)

- Whether `ALLOT` / `VALLOT` / `PICK` / `ROLL` / `BASE` should range-check their inputs against actual stack/region bounds. Currently none of them do, consistently, by choice rather than oversight.
- Whether any additional distinction like `TIB` / `SOURCE` (fixed terminal buffer vs. current input source) is needed anywhere else in the system where `EVALUATE` 's redirection might still cause a mismatch.
- `SWIH` 's throw code (`-99` in the consolidated source) was never assigned a real, deliberate value — it's a placeholder for "some hardware trap occurred," not a considered ANS-style code.
- `REPLACES` / `SUBSTITUTE` remain single-slot (one registered name/value pair, overwritten by the next `REPLACES` call) rather than a true multi-entry table. A real table needs its own storage layout and lookup structure — a deliberate scope decision made when `SUBSTITUTE` was completed, not an oversight.

What's solid (for reference, not action)

Stack manipulation (Core + Core Ext + return-stack transfer), all arithmetic (single + double + mixed precision), logic, comparison (single + double), full control flow including `CASE` , all defining words including `DEFER` / `MARKER` / `VALUE` / `TO` (`VALUE` now correctly targeting mutable space), memory and string operations (including a completed `SUBSTITUTE`), `SOURCE` / `EVALUATE` / `REFILL` mechanics, and the Tools word set (`.S` / `WORDS` / `DUMP` , with `DUMP` 's edge-case bug fixed) are complete and were traced/verified against concrete cases during the build, not merely asserted correct.